TechniView

PURPOSE

LeetCode-style interview practice, tied to coursework

PROJECT GOAL

Create a web-based learning platform
for technical interview questions.

The Team Behind TechniView

TEAM CONTACTS

Derek Corniello corniedj@mail.uc.edu **Backend + runner**

Ryan Sippy sippyrd@mail.uc.edu **Frontend**

Jason Bellerjeau bellerjn@mail.uc.edu **Data + database**

Faculty advisor: Dr William Hawkins III

Project Abstract

TechniView links **technical interview practice** to coursework in one platform, giving students a structured way to prepare alongside class assignments.

The **150-problem TechniView Trail** sequences Python problems by technique and prerequisite, helping students choose what to practice next.

Students submit code, see **per-test feedback**, and track progress across practice and assigned work.

Professors build sets, publish dated assignments, and review **class analytics** to spot where students struggle.

React, FastAPI, MySQL, and an isolated **go-judge runner** power the platform. CI checks code, integration, and production images.

A voluntary pilot pairs a usability survey with **de-identified activity data** to study learning and engagement; IRB requirements govern collection and sharing.

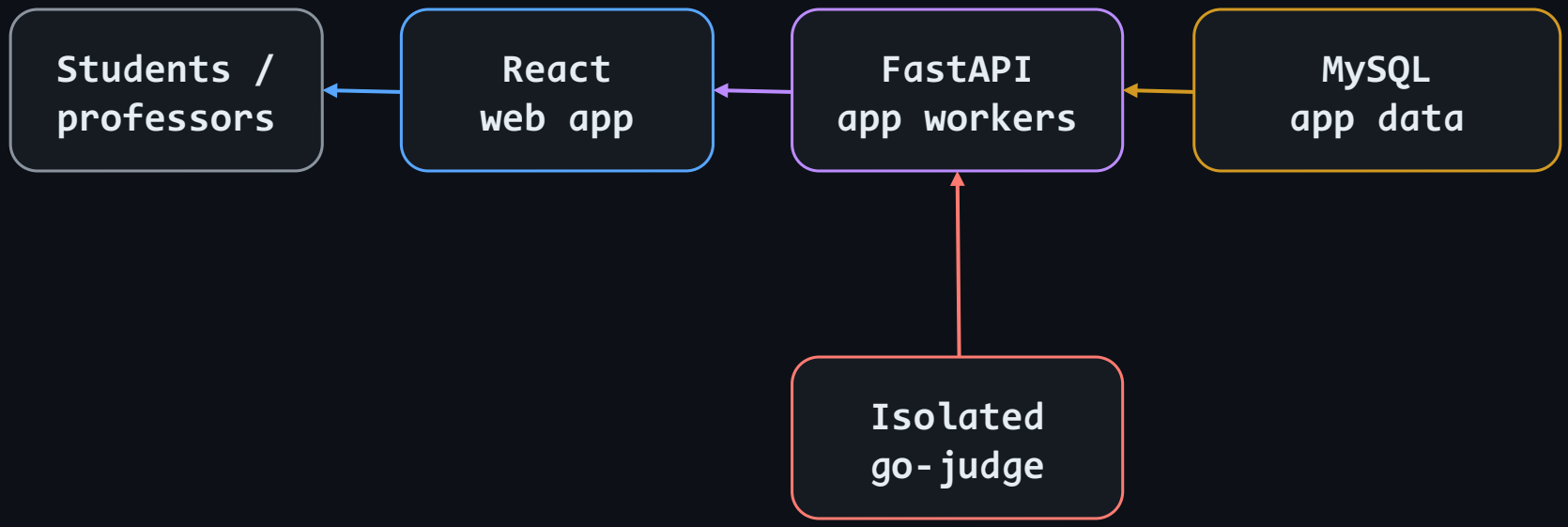
User Stories and System Design

USER STORIES

Student: practice interview problems online.

Professor: build sets, assign work, track results.

CURRENT SYSTEM DESIGN



TechniView Trail, assignments, grading, progress.

References: US-01, US-02, US-03, REQ-01, REQ-02, REQ-03, REQ-04, NFR-02

Major Project Constraints

- **Cost:** Free with existing campus hosting.
- **Security:** Student signup/access restricted; runner isolated.
- **Privacy:** Follow FERPA for course data.
- **Quality:** Verify accessibility, load, and uptime.

References: CON-01, CON-02, REQ-05, NFR-02, NFR-03, NFR-04

Project Progress

- **Python solutions** and reference cases checked.
- **Backend API** tests cover core workflows.
- **Frontend tests** cover student and instructor views.
- **Integration checks** use MySQL and go-judge.
- **Sandbox tests** exercise real submissions.
- **Production images** build in CI.

Expected End-of-Term Accomplishments

- Test deployed workflows end to end.
- Run a voluntary pilot after IRB review.
- Share study findings at Expo.

References: CON-03, CON-04, NFR-01, NFR-03

Division of Work

- **Derek:** backend, runner, deployment.
- **Ryan:** frontend, course workflows.
- **Jason:** schema, catalog, analytics.

Expected Expo Demo

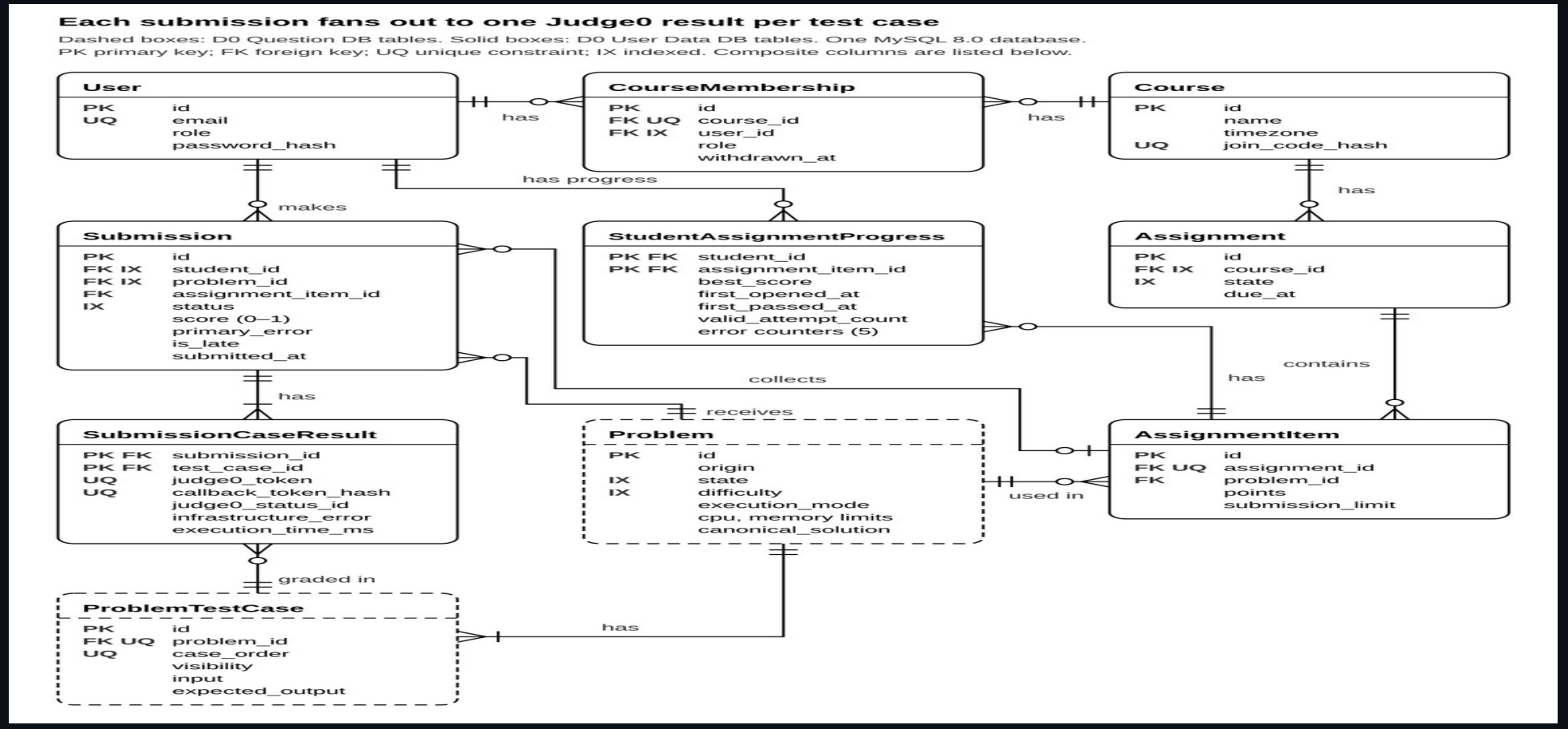
- Student solves Python; sees test results.
- **TechniView Trail** suggests the next technique.
- **Professor publishes** sets and assignments.
- **Research findings** show pilot results.
- **Recordings/screenshots:** shown and ready as fallback.

References: US-01, US-03, REQ-01, REQ-04, CON-03, CON-04

Questions?

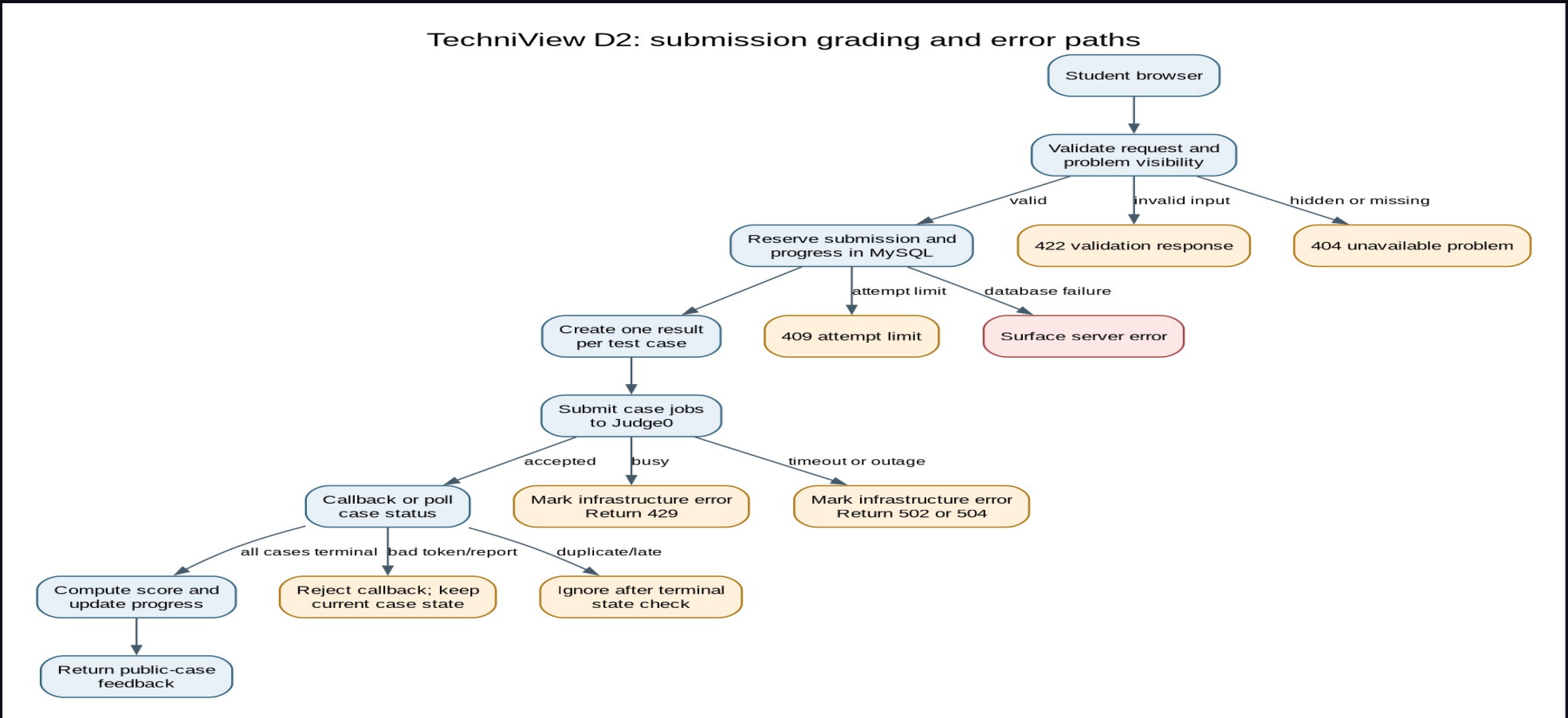
D1 Data Model | Backup

HISTORICAL DESIGN REFERENCE



D2 Grading Flow | Backup

HISTORICAL DESIGN REFERENCE



AI Use Acknowledgment

AI was used to assist with slide style and formatting.

The team is responsible for the technical content and claims.